R Basics

Foundation for Statistical Computing

Ethiopian Statistics Service (ESS)

September 18 to 20, 2026

Comprehensive Reference Guide

Modules Covered

- **Getting Started** with R & RStudio Setup
- **Core Data Types** and Vectorized Operations
- **Structured Data**: Matrices, Lists, Data Frames
- **Control Flow** & Custom Functions
- **Functional Programming** with Apply Family
- **Data Import/Export** and Tidyverse
- **Visualizations**, Debugging, Optimization

Quick Reference

This guide provides a comprehensive foundation for statistical computing with R. Each module builds upon the previous.

Assignment Operators

Standard Assignment (Preferred)

```r
x <- 10          # Standard assignment
```

Alternative Assignment (Discouraged)

```r
y = 20           # Alternative assignment
```

Pro Tip

Use <- consistently for assignment. It distinguishes assignment from argument specification and is the standard in the R community.

Variable Naming Conventions

Recommended: snake_case

```
my_variable <- 5
mean_value <- 10
total_sum <- 100
```

Alternative: camelCase

```
myVariable <- 5
meanValue <- 10
totalSum <- 100
```

Hidden Objects

```
.hidden_var <- 5  # Not displayed in ls()
```

Object Management

Environment Operations

```r
ls()              # List all objects
rm(x)             # Remove specific object
rm(list = ls())   # Clear entire workspace
```

Saving and Loading

```r
saveRDS(df, file = "data.rds")    # Save object
df_loaded <- readRDS("data.rds") # Load object
```

Help System

```r
?mean                       # Help for mean()
help.search("regression") # Search help
```

Atomic Data Types

Type	Example	Check Function
Numeric	3.14, 42	`is.numeric()`
Integer	42L	`is.integer()`
Character	"Hello"	`is.character()`
Logical	TRUE, FALSE	`is.logical()`
Factor	factor("A")	`is.factor()`

Type Coercion Hierarchy

logical → **integer** → **numeric** → **character**

Type Coercion Examples

Implicit Coercion

```r
# All become character
c(TRUE, 2, "three")  # "TRUE", "2", "three"

# Becomes numeric
c(FALSE, 3)          # 0, 3

# Logical to numeric
sum(TRUE, FALSE, TRUE)  # 2
```

Explicit Coercion

```r
as.numeric("3.14")    # 3.14
as.character(42)      # "42"
as.logical(1)         # TRUE
as.integer(3.9)       # 3 (truncates)
```

Creating Vectors

Vector Creation Methods

```r
# Concatenate
v <- c(10, 20, 30, 40, 50)

# Sequence
seq(1, 10, by = 2)    # 1, 3, 5, 7, 9

# Replicate
rep(5, times = 3)     # 5, 5, 5

# Combine with rep
rep(c(1, 2), each = 3)   # 1, 1, 1, 2, 2, 2
```

Vectorized Operations

Element-wise Operations

```r
v <- c(10, 20, 30, 40, 50)
v + 5            # 15, 25, 35, 45, 55
v * 2            # 20, 40, 60, 80, 100
v ^ 2            # 100, 400, 900, 1600, 2500
sqrt(v)          # 3.16, 4.47, 5.48, 6.32, 7.07
```

Key Insight

Vectorized operations are not just syntactically cleaner — they are much faster than explicit loops in R.

Vector Subsetting

Subsetting Methods

```r
v <- c(10, 20, 30, 40, 50)

# By position
v[3]              # 30
v[1:3]            # 10, 20, 30
v[c(1, 3, 5)]     # 10, 30, 50

# By condition
v[v > 25]         # 30, 40, 50
v[v %% 2 == 0]    # Even numbers

# By name (if named)
names(v) <- c("a", "b", "c", "d", "e")
v["c"]            # 30
```

Statistical Summaries

Summary Functions

```r
v <- c(10, 20, 30, 40, 50)

length(v)     # 5
sum(v)        # 150
mean(v)       # 30
sd(v)         # 15.81
var(v)        # 250
median(v)     # 30
```

Logical Operations

```r
any(v > 25)       # TRUE
all(v > 0)        # TRUE
which(v == 30)    # 3
v %in% c(20, 40) # FALSE, TRUE
, FALSE, TRUE, FALSE
```

Creating Data Frames

Data Frame Construction

```
df <- data.frame(
Name = c("Alice", "Bob", "Charlie", "David"),
Age = c(25, 30, 35, 28),
Score = c(88.5, 92.0, 85.5, 90.0),
Passed = c(TRUE, TRUE, FALSE, TRUE)
)
```

Exploring Data Frames

```
str(df)          # Structure
head(df)         # First 6 rows
tail(df)         # Last 6 rows
names(df)        # Column names
nrow(df)         # Number of rows
ncol(df)         # Number of columns
```

Subsetting Data Frames

Subsetting Methods

```r
# By position
df[1, 2]              # Row 1, column 2

# By column
df[, "Age"]           # Returns vector
df$Age                # Preferred method

# Filtering rows
df[df$Age > 25, ]     # Age > 25

# Selecting specific columns
df[1:2, c("Name", "Score")]  # First 2 rows
```

Important

R uses **1-based indexing** — counting starts at 1, not 0!

Manipulating Data Frames

Adding and Modifying

```r
# Add new column
df$Grade <- c("B", "A", "B", "A")

# Conditional column
df$Age_Group <- ifelse(df$Age < 30, "Young", "Senior")

# Modify existing column
df$Score <- df$Score * 1.05  # 5% bonus

# Remove column
df$Temp <- NULL  # Remove column
```

Using `transform`

```r
df <- transform(df,
Score_Adjusted = Score * 1.05,
Category = ifelse(Score >= 90, "High", "Low")
)
```

Reading and Writing Data

Standard CSV Operations

```r
# Reading
df <- read.csv("data.csv")

# Writing
write.csv(df, "output.csv", row.names = FALSE)
```

Package-based Imports

```r
library(readr)
df_fast <- read_csv("data.csv")  # Faster

library(readxl)
df_excel <- read_excel("data.xlsx", sheet = 1)
```

Use `readr::read_csv()` instead of `read.csv()` for faster parsing and better type inference.

Conditional Statements

if/else Structure

```r
x <- 10

if (x > 5) {
  print("x is greater than 5")
} else if (x == 5) {
  print("x equals 5")
} else {
  print("x is less than 5")
}
```

Vectorized Conditionals

```r
# ifelse() for vectors
y <- c(3, 7, 2, 9, 5)
ifelse(y > 5, "High", "Low")  # "Low", "High", "Low", "High", "Low"
```

Loops in R

For Loop

```r
for (i in 1:3) {
  print(i^2)
}
# 1, 4, 9
```

While Loop

```r
count <- 1
while (count <= 3) {
  print(count)
  count <- count + 1
}
# 1, 2, 3
```

Performance Note

Avoid loops when vectorized alternatives exist. Vectorized operations are significantly faster and more concise.

Writing Custom Functions

Basic Function Syntax

```r
calc_power <- function(base, exponent = 2) {
  result <- base^exponent
  return(result)
}

calc_power(4)       # Default exponent = 2 => 16
calc_power(2, 3)    # Custom exponent = 3 => 8
```

Multiple Arguments

```r
calculate_stats <- function(x, na.rm = TRUE, trim = 0) {
  if (!is.numeric(x)) stop("Input must be numeric")
  list(
  mean = mean(x, na.rm = na.rm, trim = trim),
  sd = sd(x, na.rm = na.rm),
  n = length(x)
  )
}
```

Function Best Practices

Anonymous Functions

```r
# Apply anonymous function
sapply(1:5, function(x) x^2)  # 1, 4, 9, 16, 25

# Using with apply
apply(matrix(1:9, 3, 3), 2, function(x) x / sum(x))
```

Best Practices Checklist

- ✔ Use **snake_case** for function names
- ✔ Document arguments and return values
- ✔ Keep functions single-purpose and modular
- ✔ Include input validation (`stop()` for errors)

Functional Programming with Apply

apply() — For Matrices/Data Frames

```r
m <- matrix(1:12, nrow = 3, ncol = 4)

apply(m, 1, sum)    # Row sums
apply(m, 2, mean)   # Column means
apply(m, 2, sd)     # Column SDs
```

lapply() & sapply() — For Lists/Vectors

```r
# lapply returns a list
lapply(1:3, function(x) x^2)

# sapply simplifies output
sapply(1:3, function(x) x^2)  # Vector: 1, 4, 9
```

Apply Family Examples

tapply() — Group Operations

```r
# Mean MPG by cylinder count
tapply(mtcars$mpg, mtcars$cyl, mean)

# 4-cyl: 26.66, 6-cyl: 19.74, 8-cyl: 15.10
```

mapply() — Multiple Arguments

```r
# Element-wise addition of two vectors
mapply(function(x, y) x + y, c(1, 2, 3), c(4, 5, 6))
# 5, 7, 9
```

Apply Family Examples - Cont.

Performance Comparison

```
# Loop vs Apply vs Vectorized
result_loop <- numeric(5)
for (i in 1:5) result_loop[i] <- i^2

result_apply <- sapply(1:5, function(x) x^2)
result_vec <- (1:5)^2  # Fastest!
```

Base Visualization

Histogram

```
hist(rnorm(100),
main = "Histogram",
col = "steelblue",
border = "white")
```

Scatter Plot

```
plot(1:10, 1:10,
main = "Scatter Plot",
pch = 19,
col = "darkred")
```

Boxplot

```
boxplot(mpg ~ cyl,
data = mtcars,
main = "MPG by Cylinders",
col = "lightgray")
```

Multiple Plots

Grid Layout

```r
par(mfrow = c(1, 3))  # 1 row, 3 columns

hist(mtcars$mpg, main = "MPG", col = "steelblue")
boxplot(mtcars$mpg, main = "Boxplot", col = "orange")
plot(mtcars$wt, mtcars$mpg, main = "MPG vs Weight", pch = 19)

par(mfrow = c(1, 1))  # Reset layout
```

Color Palettes

```r
colors()  # List all available colors
# Use named colors or hex codes: "#FF5733"
```

Debugging Tools

Common Debugging Functions

```
# Traceback after error
traceback()

# Interactive debugging
debug(my_function)    # Step through function
undebug(my_function) # Remove debugging

# Show warnings immediately
options(warn = 1)
```

Browser Function

```
my_function <- function(x) {
  browser()  # Pause execution here
  result <- x^2
  return(result)
}
```

Performance Optimization

Timing Code

```r
# Measure execution time
system.time({
  x <- runif(1e6)
  y <- sqrt(x)
})

# Compare approaches
system.time(for(i in 1:1000) sqrt(i))
system.time(sapply(1:1000, sqrt))
system.time(sqrt(1:1000))  # Fastest!
```

Memory Management

```r
gc()                # Garbage collection
object.size(x)      # Memory size of object
```

Code Style Checklist

☑	**Naming Conventions**
✔	Use clear variable names with `snake_case`
✔	Function names should be verbs: `calc_mean()`
✔	Use UPPERCASE for constants: `MAX_VALUE`
☑	**Performance**
✔	Prefer vectorization over explicit `for` loops
✔	Use `apply()` family for functional iteration
✔	Pre-allocate vectors when loops are necessary

Code Style Checklist: Cont.

	Data Handling
✔	Always handle missing data with `na.rm = TRUE`
✔	Use `str()` and `summary()` to inspect data
✔	Check data types with `is.*()` functions
	Code Organization
✔	Modularize complex operations into single-purpose functions
✔	Comment code with # for clarity
✔	Use ### for section headers

Key Takeaways

1. **Master vectorization** to write fast, idiomatic R code.
2. **Leverage data frames and Tidyverse** for clean data pipelines.
3. **Use function abstraction** to build reproducible workflows.
4. **Always handle missing values** explicitly.
5. **Document your code** — future you will thank you.

Remember

- **1-based Indexing**: R starts counting at 1
- **Missing Values (NA)**: Use `na.rm = TRUE`
- **Assignment**: Use <- for assignment
- **Vectors**: The fundamental building block of R

Thank You!



Happy Coding with R!

Appreciate the scene Behind Statistics!!!
